

Practica 3

```

void scan_letra(char *); ← Escanea letra y comprueba (A-Z).
void scan_dig(char *, unsigned); ← Escanea dig comprueba (0-9) printa dígito que
                                esta siendo escaneado (0-2).
void scan_DNI(unsigned *); ← Llama a scan_dig la pasa char dig y unsigned posición
                                del bucle y *dni = *dni * 10 + (dig - '0').
unsigned resto_DNI(unsigned); ← return dni % 23.
char letra_calculada(unsigned); ← switch con las letras return.
void print_menu();
void scan_opcion(int *);
    
```

Practica 4

```

void scan_dig(char *, unsigned);
void scan_DNI(unsigned *);
unsigned resto_DNI(unsigned);
void scan_letra(char *);
char letra_calculada(unsigned);
void validar_letra_DNI(char, unsigned);
    
```

Todo el main con memoria dinámica:

```

unsigned *dni;
dni = (unsigned *) malloc(sizeof(unsigned));
free(dni);
    
```

Practica 5

```

void rand_dig(char *);
void rand_DNI(unsigned *);
unsigned resto_DNI(unsigned);
char letra_calculada(unsigned);
void rand_DNIs(unsigned DNIs[N], char LETRAS[N]); ← Llama N veces a
                                                        rand_DNI y calcula las
                                                        letras.
void print_DNIs(unsigned DNIs[N], char LETRAS[N]);
void calcular_frecuencias(char [N], unsigned [26], float [26]); ← Recorre (j < 26)
                                                                    letra = 'A' + j y
                                                                    la compara con todas las
                                                                    LETRAS[i] en otro bucle
                                                                    y va guardando Frel[j]++ y Fabs[j] = float(Frel[j])
void printf_frecuencias(unsigned [26], float [26]);
void print_barra(float); ← Printa todos (j < 26)
                            los letras 'A' + j y llama a print_barra(Fabs[j] * 100)
    
```

Con fabs = Fabs[j] * 100
 printa tantas asteriscos
 como ocurrencia de esa
 letra haya.

Practica 6

`void rand-dig(char *);` ← `*digito = '0' + (rand() % 10);`
`void rand-str-DNI(char [9+1]);` ← Se llama a las funciones `rand-dig`, `resto-DNI`, letra calculada y se monta el string `dni-str`.
`unsigned resto-DNI(unsigned);` ← `return dni % 23;`
`char letra-calculada(unsigned);` ← Array `letra-cal[ ] = {'T', 'R', ..., 'E'}`
`return letra-cal[resto-dni];`
`void rand-strings-DNIs(char [N][9+1]);` ← Se llama a `rand-str-DNI` N veces, strcpy a DNIs.
`void print-strings-DNIs(char [N][9+1]);` ← N veces `printf("%s\t", DNIs[i]);`
`void print-menu();` ← Uso de `getch` (`conio.h`) sin mostrar por pantalla, devuelve el char como int `getch - '0' = n° real`.
`void buscar-substring(char [N][9+1]);` (`opc=1`) ← `strchr` de la letra escaneada en esta opción.
`void buscar-letra(char [N][9+1]);` (`opc=2`) ← `strstr` de la cadena escaneada en esta opción.

Practica 7

`void rand-dig(char *);`
`unsigned resto-DNI(unsigned);`
`char letra-calculada(unsigned);`
`void rand-DNI(unsigned *);` ← Llama a `rand-dig`; `*dni = (*dni * 10) + (digito - '0');`
`void rand-DNIs(unsigned [N], char [N]);` ← Llama a `rand-DNI`, `resto-DNI`, letra calculada N veces.
`void print-DNIs(unsigned [N], char [N]);` ← Printa N veces `DNIs[i]` `LETRAS[i]`.
`void swap-unsigned(unsigned *, unsigned *);` ← Se cambian las posiciones mayor y menor con una variable temporal (maior).
`void swap-char(char *, char *);` ← Mismo cambio, con las letras.
`void bubbleSort(unsigned [N], char [N]);`

```

for(int i=0; i<N; i++){
  for(int j=0; j<N-i-1; j++){
    if(DNIs[j] > DNIs[j+1]){
      swap-unsigned(&DNIs[j], &DNIs[j+1]);
      swap-char(&LETRAS[j], &LETRAS[j+1]);
    }
  }
}
  
```

En el main y las funciones swap se utiliza asignación dinámica de memoria.

Practica 8

```

void rand-dig(char *);
void rand-str-DNI(char [9+1]);
unsigned resto-DNI(unsigned);
char letra-calculada(unsigned);
void rand-strings-DNIs(char [N][9+1]);
void print-strings-DNIs(char [N][9+1]);
void strings-swap(char mayor [9+1], char menor [9+1]);
void strings-bubbleSort(char DNIs [N][9+1]);
    for(int i=0; i<N; i++){
        for(int j=0; j<N-1-i; j++){
            if(memcmp(DNIs[j], DNIs[j+1], sizeof(DNIs[j])) > 0){
                strings-bubbleSort(DNIs[j], DNIs[j+1]);
            }
        }
    }

```

Con un array temporal
comblando de lugar con
memmove(temp, mayor, sizeof(temp))

copia mayor en
temp con el tamaño
de temp (lo copia todo)

memcmp(a, b, n)
compara los primeros n bytes
0 son iguales
<0 a > b
>0 b > a

Practica 9

```

void rand-dig(char *);
void rand-str-DNI(char [9+1]);
unsigned resto-DNI(unsigned);
char letra-calculada(unsigned);
void rand-DATE(fecha DATE *);
unsigned es-fecha-valida(DATE);
int dias[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
Año bisieto → if(! fecha.ano % 4 == 0 && (! fecha.ano % 100 != 0 || fecha.ano % 400 == 0))
    dias[2] = 29;
if(! fecha.mes < 1 || fecha.mes > 12 || fecha.dia < 1 || fecha.dia > dias[fecha.mes])
    return 1;
void rand-nom-comp(char [20+1]);
void rand-ALUMNO(struct ALUMNO);
void print-DATE(DATE);
void print-ALUMNO(struct ALUMNO);

```

Usa es-fecha-valida para comprobar si los días
con el mes y el año son posibles y si no
lo son los vuelve a crear.

Del array nombres y apellidos copia aleatoriamente
y los concatena en nom-ape.

Llama a las funciones rand-str-DNI, rand-nom-
comp, rand-DATE y les pasa los elementos
de la estructura

Manda a generar una fecha.

Practica 40

```
void rand_dig(char *);
void rand_str_DNI(char [A+1]);
unsigned resto_DNI(unsigned);
char letra_calculada(unsigned);
void rand_DATE(DATE *);
unsigned es_fecha_valida(DATE);
void rand_nom_comp(char [20+1]);
void rand_ALUMNO(struct ALUMNO);
void print_DATE(DATE);
void print_ALUMNO(struct ALUMNO);
void all_swap(struct ALUMNO * activo, struct ALUMNO * sig);
void all_bubbleSort(struct ALUMNO alumno [A], unsigned opc);
    for(int i=0; i<A; i++) {
        if(opc==2)
            for(int j=0; j<A-1-1; j++) { nom_comp nom_comp
                if(strcmp(alumno[j].DNI, alumno[j+1].DNI) > 0) {
                    all_swap(&alumno[j], &alumno[j+1]);
                }
            }
        }
    }
void fprint_DATE(FILE*, DATE);
void fprint_ALUMNO(FILE*, struct ALUMNO);
```

Con una estructura temporal
cambia el orden de uno
con el otro.
opc = 1 ordena por DNI
opc = 2 ordena por letra.

} Para escribir en el
fichero binario.